



DATA ENGINEERING (CLOUD ENGINEERING) - DAY-WISE COMPLETE PLAN

June 1 - August 1, 2026 | 61-Day Master Plan



OVERVIEW

Your Data Engineering Track:

- 2 hours/day focused study (8:30 AM - 10:30 AM)
- Build a complete Spark pipeline for production
- Deploy on AWS with Docker
- Publish on Medium
- Create professional GitHub portfolio

Final Outcome:

- ✓ Spark pipeline (data processing)
 - ✓ Docker container (containerization)
 - ✓ AWS EC2 deployment (cloud)
 - ✓ Blog post (documentation)
 - ✓ Professional portfolio (GitHub)
-



PHASE 1: JUNE (Days 1-30) - BUILDING FOUNDATION

WEEK 1 (June 1-7): AWS & PYTHON SETUP ⚙️

DAY 1 (June 1) - LAUNCH: AWS & BASICS

Duration: 120 mins (8:30 AM - 10:30 AM)

MORNING TASK:

Task 1: AWS Account Setup (30 mins)

- Go to: `aws.amazon.com`
- Click: Free Tier Sign Up
- Enter: Email, password, payment info
- Verify: Phone number
- Select: Personal account type
- Complete: Basic information
- Result: AWS account created ✓

Task 2: AWS Console Exploration (20 mins)

- └─ Log in to AWS Console
- └─ Find: EC2 service
- └─ Understand: Dashboard layout
- └─ Note: Available free tier resources
- └─ Screenshot: Dashboard (save to folder)

Task 3: Video Learning (20 mins)

- └─ Watch: "AWS Free Tier Explained"
- └─ Link: youtube.com/watch?v=3hLmDS179YE
- └─ Duration: 20 minutes
- └─ Take: Notes on free tier limits
- └─ Key Learning: What you can use free

Task 4: First Script (20 mins)

- └─ Create: hello.py
- └─ Write:

```
print("Day 1: Data Engineering Begins!")
print("AWS Account: Ready")
print("Time: 120 minutes")
```
- └─ Run: `python3 hello.py`
- └─ Result: Output shows 

Task 5: Documentation (10 mins)

- └─ Create: day_1_notes.md
- └─ Note: What you learned
- └─ Save: AWS dashboard screenshot
- └─ Commit: "Day 1: AWS account created"

LEARNING OBJECTIVES:

-  AWS account created
-  AWS console understood
-  Free tier limits known
-  First Python script runs

DELIVERABLE: AWS account + Screenshot

DAY 2 (June 2) - EC2 INSTANCE

Duration: 120 mins

MORNING TASK:

Task 1: Watch EC2 Tutorial (30 mins)

- Video: "AWS EC2 Complete Tutorial"
- Link: youtube.com/watch?v=oHIYBCYn9Ts
- Learn: Instance types, security groups, key pairs
- Take: Detailed notes

Task 2: Create EC2 Key Pair (20 mins)

- Go to: AWS Console → EC2 → Key Pairs
- Click: Create Key Pair
- Name: my-learning-keypair
- Format: .pem
- Download: Save to ~/.ssh/
- Protect: `chmod 400 ~/.ssh/my-learning-keypair.pem`
- Result: Key pair ready 

Task 3: Launch EC2 Instance (40 mins)

- Go to: EC2 Dashboard → Instances
- Click: Launch Instance
- Select: Ubuntu Server 20.04 LTS (Free tier eligible)
- Instance Type: t2.micro (Free tier)
- Key Pair: my-learning-keypair
- Security Group: Allow SSH (port 22)
- Storage: 30 GB (Free tier)
- Launch: Create instance
- Wait: Instance running (shows green)

Task 4: Get Public IP (10 mins)

- Find: Instance public IP address
- Note: IP address (e.g., 54.123.45.67)
- Check: Instance status shows "running" and "2/2"
- Save: IP to notes

Task 5: Document & Commit (20 mins)

- Update: day_2_notes.md
- Record: EC2 instance IP
- Record: Key pair location
- Add: Screenshot of running instance
- Commit: "Day 2: EC2 instance running"

LEARNING OBJECTIVES:

-  EC2 concepts understood
-  Key pair created
-  Instance launched
-  Instance running

DELIVERABLE: Running EC2 instance + IP address

DAY 3 (June 3) - SSH & LINUX BASICS

Duration: 120 mins

MORNING TASK:

Task 1: SSH Connection (20 mins)

- Open: Terminal
- Command: `ssh -i ~/.ssh/my-learning-keypair.pem ubuntu@YOUR_IP`
- Replace: YOUR_IP with your instance IP
- Type: yes (when asked about fingerprint)
- Result: Connected to EC2 
- Prompt: `ubuntu@ip-xxx:~$`

Task 2: Linux Basics (40 mins)

- Command: `pwd` (print working directory)
- Command: `ls` (list files)
- Command: `mkdir data-eng` (create folder)
- Command: `cd data-eng` (change directory)
- Command: `touch hello.txt` (create file)
- Command: `cat hello.txt` (read file)
- Command: `echo "Hello AWS" > hello.txt`
- Command: `cat hello.txt`
- Practice: Try all commands multiple times

Task 3: Update System (20 mins)

- Command: `sudo apt-get update`
- Command: `sudo apt-get upgrade -y`
- Wait: Installation completes
- Note: System updated 

Task 4: Install Python (25 mins)

- Command: `python3 --version` (check)
- Command: `sudo apt-get install python3-pip -y`
- Command: `pip3 --version`
- Command: `pip3 install --upgrade pip`
- Result: Python3 + pip ready 

Task 5: Create Project Folder (15 mins)

- Command: `mkdir -p ~/data-engineering-project`
- Command: `cd ~/data-engineering-project`
- Command: `mkdir data input output scripts`
- Command: `ls -la` (verify structure)
- Structure created 

LEARNING OBJECTIVES:

-  SSH connection successful
-  Linux commands working
-  System updated
-  Python installed
-  Project folder structure ready

DELIVERABLE: Connected EC2 instance with Python installed

DAY 4 (June 4) - S3 BUCKET & PYTHON ON AWS

Duration: 120 mins

MORNING TASK:

Task 1: Create S3 Bucket (30 mins)

- Go to: AWS Console → S3
- Click: Create Bucket
- Name: data-engineering-vikas-2026
- Region: ap-south-1 (or closest)
- Settings: Block all public access
- Create: Bucket created 
- Note: Bucket name

Task 2: Configure AWS CLI on EC2 (25 mins)

- SSH into EC2 (using Day 3 method)
- Command: `sudo apt-get install awscli -y`
- Command: `aws --version`
- Command: `aws configure`
 - Access Key: (you'll get from AWS)
 - Secret Access Key: (you'll get)
 - Region: ap-south-1
 - Format: json
- Result: AWS CLI configured 

Task 3: S3 Access Test (25 mins)

- Command: `aws s3 ls` (list buckets)
- Command: `aws s3 mb s3://data-eng-test-vikas` (create test bucket)
- Command: `echo "test data" > test.txt`
- Command: `aws s3 cp test.txt s3://data-eng-test-vikas/`
- Command: `aws s3 ls s3://data-eng-test-vikas/`
- Result: File uploaded to S3 

Task 4: Python on EC2 (25 mins)

- SSH into EC2
- Command: `pip3 install boto3` (AWS Python SDK)

```
|— Create: test_s3.py
|   |— import boto3
|   |— s3 = boto3.client('s3')
|   |— response = s3.list_buckets()
|   |— print(response['Buckets'])
|— Command: python3 test_s3.py
|— Result: S3 buckets listed via Python 
```

Task 5: Document & Commit (15 mins)

```
|— Update: day_4_notes.md
|— Record: S3 bucket names
|— Record: AWS CLI working
|— Add: Screenshots
|— Commit: "Day 4: S3 + AWS CLI ready"
```

LEARNING OBJECTIVES:

-  S3 bucket created
-  AWS CLI installed & configured
-  File uploaded to S3
-  Python boto3 working
-  AWS integration successful

DELIVERABLE: S3 bucket + AWS CLI + boto3 SDK

DAY 5 (June 5) - PANDAS DATA PROCESSING

Duration: 120 mins

MORNING TASK:

Task 1: Install Pandas (15 mins)

```
|— SSH into EC2
|— Command: pip3 install pandas
|— Command: pip3 install jupyter
|— Command: python3 -c "import pandas; print(pandas.__version__)"
|— Result: Pandas ready 
```

Task 2: Create Sample Data (20 mins)

```
|— Create: sample_data.csv
|— Content:
    order_id,customer_id,product,amount,status
    1,101,Laptop,50000,Completed
    2,102,Phone,25000,Pending
    3,103,Tablet,15000,Completed
    4,101,Watch,5000,Cancelled
```

5,104,Laptop,50000,Completed

- └─ Save: To EC2
- └─ Upload: To S3
- └─ File ready 

Task 3: Read CSV with Pandas (30 mins)

- └─ Create: read_data.py
- └─ Code:

```
import pandas as pd

# Read CSV
df = pd.read_csv('sample_data.csv')

# Display info
print(df)
print(df.info())
print(df.describe())

# Basic stats
print(f"Total orders: {len(df)}")
print(f"Total amount: {df['amount'].sum()}")
```
- └─ Run: python3 read_data.py
- └─ Result: Data displayed 

Task 4: Data Cleaning (35 mins)

- └─ Create: clean_data.py
- └─ Code:

```
import pandas as pd

df = pd.read_csv('sample_data.csv')

# Check missing values
print(df.isnull().sum())

# Remove duplicates
df = df.drop_duplicates()

# Fix data types
df['amount'] = pd.to_numeric(df['amount'])
df['order_id'] = df['order_id'].astype(int)

# Save cleaned data
df.to_csv('cleaned_data.csv', index=False)
print("Data cleaned and saved!")
```
- └─ Run: python3 clean_data.py
- └─ Result: Cleaned CSV saved 

```
Task 5: Commit & Document (20 mins)
├─ Update: day_5_notes.md
├─ Record: Pandas operations learned
├─ Add: Code snippets
└─ Commit: "Day 5: Pandas data processing"
```

LEARNING OBJECTIVES:

-  Pandas installed
-  CSV file reading
-  Data exploration (info, describe, stats)
-  Data cleaning (nulls, duplicates, types)
-  Data saving

DELIVERABLE: Cleaned CSV file + Pandas knowledge

DAY 6 (June 6) - SQL & DATABASE BASICS

Duration: 120 mins

MORNING TASK:

```
Task 1: Install SQLite (15 mins)
├─ SSH into EC2
├─ Command: sudo apt-get install sqlite3 -y
├─ Command: sqlite3 --version
└─ Result: SQLite ready 

Task 2: Learn SQL Basics (40 mins)
├─ Watch: SQL Tutorial (15 mins)
│   └─ Link: youtube.com/watch?v=zbMHLZEMKmk
├─ Learn: SELECT, WHERE, ORDER BY, GROUP BY
├─ Learn: INSERT, UPDATE, DELETE
├─ Learn: JOIN, aggregate functions
└─ Take: Notes on SQL concepts

Task 3: Create Database & Tables (30 mins)
├─ Command: sqlite3 orders.db
├─ In SQLite:
    CREATE TABLE orders (
        order_id INTEGER PRIMARY KEY,
        customer_id INTEGER,
        product TEXT,
        amount REAL,
        status TEXT
```



```
);  
  
INSERT INTO orders VALUES  
(1, 101, 'Laptop', 50000, 'Completed'),  
(2, 102, 'Phone', 25000, 'Pending'),  
(3, 103, 'Tablet', 15000, 'Completed');
```

```
|  
└─ Command: SELECT * FROM orders;  
└─ Result: Data in database 
```

Task 4: Query Data with Python (25 mins)

```
└─ Create: query_data.py
```

```
└─ Code:
```

```
import sqlite3  
  
conn = sqlite3.connect('orders.db')  
cursor = conn.cursor()  
  
# Simple query  
cursor.execute('SELECT * FROM orders')  
rows = cursor.fetchall()  
for row in rows:  
    print(row)  
  
# Aggregate query  
cursor.execute('SELECT status, COUNT(*) FROM orders GROUP BY status')  
results = cursor.fetchall()  
for status, count in results:  
    print(f"{status}: {count}")  
  
conn.close()
```

```
└─ Run: python3 query_data.py
```

```
└─ Result: Data queried 
```

Task 5: Document & Commit (10 mins)

```
└─ Update: day_6_notes.md
```

```
└─ Record: SQL commands learned
```

```
└─ Add: Database schema
```

```
└─ Commit: "Day 6: SQLite database ready"
```

LEARNING OBJECTIVES:

-  SQLite installed
-  Database created
-  Tables created with schema
-  INSERT data
-  SELECT queries

-  SELECT queries
-  GROUP BY aggregations
-  Python database connection

DELIVERABLE: SQLite database + SQL queries

DAY 7 (June 7) - WEEK 1 REVIEW & SPARK INTRO

Duration: 120 mins

MORNING TASK:

Task 1: Week 1 Review (30 mins)

- └─ Review: All Day 1-6 notes
- └─ Practice: SSH into EC2
- └─ Check: AWS console
- └─ List: S3 buckets
- └─ Review: Python scripts written
- └─ Consolidate: Understanding of basics

Task 2: Watch Spark Intro Video (30 mins)

- └─ Video: "Apache Spark Explained"
- └─ Link: youtube.com/watch?v=QV5KxAQNBz0
- └─ Learn: What is Spark
- └─ Learn: Why use Spark (not Pandas alone)
- └─ Learn: RDD vs DataFrame
- └─ Take: Comprehensive notes

Task 3: Install PySpark (30 mins)

- └─ SSH into EC2
- └─ Command: `pip3 install pyspark`
- └─ Command: `python3 -c "import pyspark; print(pyspark.__version__)"`
- └─ Verify: Installation successful
- └─ Result: PySpark ready 

Task 4: First Spark Program (25 mins)

- └─ Create: `first_spark.py`
- └─ Code:

```
from pyspark.sql import SparkSession

spark = SparkSession \
    .builder \
    .appName("FirstSpark") \
    .getOrCreate()

print(f"Spark Version: {spark.version}")
```

```
print(f"App Name: {spark.sparkContext.appName}")
```

```
spark.stop()
```

```
└─ Run: python3 first_spark.py
```

```
└─ Result: Spark working 
```

Task 5: Summary & Commit (5 mins)

```
└─ Create: WEEK1_SUMMARY.md
```

```
└─ Recap: Everything learned
```

```
└─ List: Tools installed
```

```
└─ Record: Status
```

```
└─ Commit: "Day 7: Week 1 complete! AWS+Python+Spark ready"
```

WEEK 1 CHECKPOINT:

 AWS account & EC2 instance

 S3 bucket for data storage

 Python environment ready

 Pandas for data processing

 SQLite for databases

 PySpark installed

Status:  Ready for Phase 2!

LEARNING OBJECTIVES:

-  Consolidated Week 1 learning
-  Understood Spark basics
-  PySpark installed
-  First Spark program runs

DELIVERABLE: Spark environment ready

WEEK 2 (June 8-14): SPARK FUNDAMENTALS

DAY 8 (June 8) - SPARK & DATAFRAMES

Duration: 120 mins

MORNING TASK:

Task 1: Spark Architecture (20 mins)

```
└─ Learn: Driver, Executors, Tasks
```

```
└─ Learn: RDD, DataFrame, SQL
```

```
└─ Learn: Lazy evaluation
```

```
└─ Watch: First 10 mins of PySpark tutorial
```

Task 2: Create SparkSession (15 mins)

Task 2: Create SparkSession (10 mins)
└─ Create: spark_session.py
└─ Code:

```
from pyspark.sql import SparkSession

spark = SparkSession \
    .builder \
    .appName("DataEngineeringApp") \
    .master("local[*]") \
    .getOrCreate()

sc = spark.sparkContext
sc.setLogLevel("WARN")
print(f"Spark Version: {spark.version}")
```

└─ Run: python3 spark_session.py

└─ Result: SparkSession created 

Task 3: Create DataFrame from List (30 mins)

└─ Create: create_dataframe.py

└─ Code:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("DF").master("local[*]").getOrCreate()

# Method 1: From list
data = [
    (1, "Alice", 25000),
    (2, "Bob", 30000),
    (3, "Charlie", 28000)
]

df = spark.createDataFrame(data, ["id", "name", "salary"])
df.show()
df.printSchema()
```

└─ Run: python3 create_dataframe.py

└─ Result: DataFrame displayed 

Task 4: Read CSV in Spark (35 mins)

└─ Create: read_csv_spark.py

└─ Code:

```
from pyspark.sql import SparkSession
```

```
spark =
SparkSession.builder.appName("ReadCSV").master("local[*]").getOrCreate()
```

```
# Read CSV
```

```
df = spark.read \
```

```
    .option("header", "true") \
```

```
    .option("interSchema", "true") \
    .csv("sample_data.csv")
```

```
# Display
df.show()
df.printSchema()
df.describe().show()

# Basic stats
print(f"Total records: {df.count()}")
print(f"Total columns: {len(df.columns)}")
```

└─ Run: python3 read_csv_spark.py

└─ Result: CSV read successfully 

Task 5: Commit & Document (20 mins)

└─ Update: day_8_notes.md

└─ Record: Spark concepts

└─ Add: Code examples

└─ Commit: "Day 8: Spark DataFrames created"

LEARNING OBJECTIVES:

-  SparkSession created
-  DataFrame creation from data
-  CSV reading with Spark
-  Schema understanding
-  Basic data exploration

DELIVERABLE: Spark DataFrame operations

DAY 9 (June 9) - SPARK TRANSFORMATIONS PART 1

Duration: 120 mins

MORNING TASK:

Task 1: Learn select() Operation (25 mins)

└─ Watch: Spark select tutorial (10 mins)

└─ Concept: Select specific columns

└─ Code example:

```
df.select("name", "salary").show()
```

```
df.select("id", "name").show()
```

```
df.select(df.id, df.salary).show()
```

└─ Understand: Different select syntaxes

Task 2: Learn filter() Operation (25 mins)

└─ Concept: Filter rows based on condition

└─ Code example:

```
df.filter(df.salary > 25000).show()
df.filter((df.id == 1) | (df.id == 2)).show()
df.filter(~(df.name == "Bob")).show()
```

└─ Understand: Conditions and logic

Task 3: Hands-on Transformations (50 mins)

└─ Create: transformations_1.py

└─ Code:

```
from pyspark.sql import SparkSession

spark =
SparkSession.builder.appName("Transform1").master("local[*]").getOrCreate()
```

```
df = spark.read \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .csv("sample_data.csv")
```

Select specific columns

```
print("Select: order_id, product, amount")
df.select("order_id", "product", "amount").show()
```

Filter by status

```
print("Filter: status == 'Completed'")
df.filter(df.status == "Completed").show()
```

Filter by amount

```
print("Filter: amount > 20000")
df.filter(df.amount > 20000).show()
```

Combine select and filter

```
print("Select and Filter combined")
df.select("product", "amount") \
    .filter(df.amount > 20000) \
    .show()
```

└─ Run: python3 transformations_1.py

└─ Result: Multiple transformations working 

Task 4: Save Results (15 mins)

└─ Modify: Add save operation

└─ Code:

```
# Save filtered results
filtered = df.filter(df.status == "Completed")
```

```
filtered.write \
```

```
.mode("overwrite") \  
.option("header", "true") \  
.csv("output/completed_orders")
```

└─ Verify: Files created in output folder

└─ Result: Data saved 

Task 5: Commit & Document (5 mins)

└─ Update: day_9_notes.md

└─ Commit: "Day 9: Spark select() & filter() working"

LEARNING OBJECTIVES:

-  select() columns
-  filter() rows with conditions
-  Combine multiple operations
-  Save results to CSV

DELIVERABLE: Spark transformation code

DAY 10 (June 10) - SPARK TRANSFORMATIONS PART 2

Duration: 120 mins

MORNING TASK:

Task 1: Learn withColumn() (20 mins)

└─ Concept: Add/modify columns

└─ Code example:

```
from pyspark.sql.functions import col  
df.withColumn("salary_in_lakhs", col("salary") / 100000)
```

└─ Understand: Creating calculated columns

Task 2: Learn groupBy() & agg() (30 mins)

└─ Concept: Group by category, aggregate values

└─ Code example:

```
df.groupBy("status").count().show()  
df.groupBy("status").agg({"amount": "sum"}).show()
```

└─ Understand: Aggregation functions

Task 3: Hands-on Aggregations (45 mins)

└─ Create: transformations_2.py

└─ Code:

```
from pyspark.sql import SparkSession  
from pyspark.sql.functions import col, sum, count, avg, max
```

```
spark =
```

```
SparkSession.builder.appName("Transform2").master("local[*]").getOrCreate()
```

```
df = spark.read \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .csv("sample_data.csv")

# Add new column
df_with_tax = df.withColumn("tax", col("amount") * 0.18)
print("With tax column:")
df_with_tax.show()

# Group by status and count
print("Count by status:")
df.groupBy("status").count().show()

# Group by status and sum amount
print("Total amount by status:")
df.groupBy("status").agg(sum("amount")).show()

# Multiple aggregations
print("Multiple aggregations:")
df.groupBy("status").agg(
    count("*").alias("count"),
    sum("amount").alias("total_amount"),
    avg("amount").alias("avg_amount"),
    max("amount").alias("max_amount")
).show()
```

└─ Run: python3 transformations_2.py

└─ Result: Aggregations working 

Task 4: Save Aggregated Results (15 mins)

└─ Modify: Save grouped results

└─ Code:

```
results = df.groupBy("status").agg(
    sum("amount").alias("total")
)
results.write \
    .mode("overwrite") \
    .option("header", "true") \
    .csv("output/summary_by_status")
```

└─ Verify: Output files created

└─ Result: Saved 

Task 5: Commit & Document (10 mins)

└─ Update: day_10_notes.md

└─ Commit: "Day 10: GroupBy & aggregations working"

LEARNING OBJECTIVES:

-  withColumn() to create new columns
-  groupBy() for grouping
-  agg() for aggregations (count, sum, avg, max)
-  Save aggregated results

DELIVERABLE: Aggregation code

DAY 11 (June 11) - SPARK JOINS & ADVANCED

Duration: 120 mins

MORNING TASK:

Task 1: Learn Joins (30 mins)

- └─ Concept: Inner, Left, Right, Full joins
- └─ Create sample data:

```
customers = [(101, "Alice"), (102, "Bob")]
orders = [(1, 101, 50000), (2, 102, 30000)]
```
- └─ Understand: How joins work

Task 2: Implement Joins (45 mins)

- └─ Create: joins_example.py
- └─ Code:

```
from pyspark.sql import SparkSession

spark =
SparkSession.builder.appName("Joins").master("local[*]").getOrCreate()

# Create DataFrames
customers_data = [
    (101, "Alice"),
    (102, "Bob"),
    (103, "Charlie")
]
customers = spark.createDataFrame(customers_data, ["cust_id", "cust_name"])

orders_data = [
    (1, 101, 50000),
    (2, 102, 30000),
    (3, 101, 20000)
]

orders = spark.createDataFrame(orders_data, ["order_id", "cust_id", "amount"])
```

```

orders = spark.createDataFrame(orders_data, ["order_id", "cust_id",
"amount"])

# Inner join
print("Inner Join:")
joined = customers.join(orders, customers.cust_id == orders.cust_id,
"inner")
joined.show()

# Left join
print("Left Join:")
left_joined = customers.join(orders, customers.cust_id == orders.cust_id,
"left")
left_joined.show()
└─ Run: python3 joins_example.py
└─ Result: Joins working ✅

```

Task 3: Advanced Operations (25 mins)

```

└─ Create: advanced_ops.py
└─ Code:
    from pyspark.sql.functions import col, when, round, year

    # Case when
    df = df.withColumn(
        "order_category",
        when(col("amount") > 40000, "High")
        .when(col("amount") > 20000, "Medium")
        .otherwise("Low")
    )

    # Round values
    df = df.withColumn("rounded_amount", round(col("amount"), -3))

    # Distinct
    df.select("status").distinct().show()

    # OrderBy
    df.orderBy(col("amount").desc()).show()
└─ Run: python3 advanced_ops.py
└─ Result: Advanced ops working ✅

```

Task 4: Combine Everything (15 mins)

```

└─ Create: complete_pipeline.py
└─ Combine:
    - Read CSV
    - Filter
    - Transform with withColumn
    - Group and aggregate

```

Group and Aggregate

- Save results

└─ Result: Complete pipeline 

Task 5: Commit & Document (5 mins)

└─ Commit: "Day 11: Joins & advanced Spark operations"

LEARNING OBJECTIVES:

-  Joins (inner, left, right)
-  Case when conditions
-  Distinct values
-  Order by sorting
-  Complete data pipeline

DELIVERABLE: Complete Spark pipeline

DAY 12 (June 12) - BUILD YOUR PROJECT PIPELINE

Duration: 120 mins

MORNING TASK:

Task 1: Design Your Pipeline (15 mins)

└─ Goal: Sales Analysis Pipeline

└─ Input: orders.csv

└─ Processing:

└─ Data cleaning

└─ Data validation

└─ Transformations

└─ Aggregations

└─ Output:

└─ sales_by_status.csv

└─ sales_by_product.csv

└─ daily_summary.csv

└─ Plan: Document steps

Task 2: Prepare Clean Data (20 mins)

└─ Create: sample_data.csv (real sales data)

└─ Content:

order_id,order_date,customer_id,product,amount,status

1001,2024-06-01,101,Laptop,50000,Completed

1002,2024-06-01,102,Phone,25000,Completed

1003,2024-06-01,103,Tablet,15000,Pending

1004,2024-06-02,101,Watch,5000,Completed

... (add more realistic data)


```

        sum("amount").alias("total_sales"),
        avg("amount").alias("avg_sales")
    )
sales_by_status.show()

# Step 6: Sales by Product
print("Step 6: Sales by product...")
sales_by_product = df.groupBy("product").agg(
    count("order_id").alias("order_count"),
    sum("amount").alias("total_sales"),
    sum("tax").alias("total_tax")
).orderBy(col("total_sales").desc())
sales_by_product.show()

# Step 7: Save Results
print("\nStep 7: Saving results...")
sales_by_status.write \
    .mode("overwrite") \
    .option("header", "true") \
    .csv("output/sales_by_status")

sales_by_product.write \
    .mode("overwrite") \
    .option("header", "true") \
    .csv("output/sales_by_product")

print("Pipeline completed successfully!")
spark.stop()

```

└─ Run: python3 sales_pipeline.py

└─ Result: Pipeline works! Output files created 

Task 4: Verify Output Files (20 mins)

└─ Check: output/sales_by_status/

└─ Check: output/sales_by_product/

└─ View: Files created successfully

└─ Save: Copy outputs to S3

Task 5: Commit & Document (5 mins)

└─ Update: day_12_notes.md

└─ Record: Pipeline complete

└─ Commit: "Day 12: Sales pipeline working!"

WEEK 2 CHECKPOINT:

-  Spark DataFrames
-  Transformations (select, filter, withColumn)
-  Aggregations (groupBy, sum, count, avg)

✓ Joins working
✓ Complete pipeline built
Status: 🟢 Core Spark skills ready!

LEARNING OBJECTIVES:

- ✓ Complete data pipeline
- ✓ Data cleaning
- ✓ Data validation
- ✓ Multiple outputs
- ✓ Production-like code

DELIVERABLE: Working Spark pipeline + Output files

DAY 13 (June 13) - OPTIMIZATION & ERROR HANDLING

Duration: 120 mins

MORNING TASK:

Task 1: Code Optimization (30 mins)

- └─ Concept: Efficient Spark code
- └─ Improvements:
 - └─ Avoid unnecessary shuffles
 - └─ Use cache() wisely
 - └─ Partitioning
 - └─ Broadcast variables
- └─ Learn: Best practices

Task 2: Implement Error Handling (35 mins)

- └─ Create: optimized_pipeline.py
- └─ Code:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, when, coalesce

try:
    spark = SparkSession \
        .builder \
        .appName("OptimizedPipeline") \
        .master("local[*]") \
        .getOrCreate()

    # Handle missing values
    df = spark.read \
        .option("header", "true") \
```

```

.option("inferSchema", "true") \
.csv("sample_data.csv")

# Fill null values
df = df.fillna({
    "amount": 0,
    "status": "Unknown"
})

# Or use coalesce
df = df.withColumn(
    "amount",
    coalesce(col("amount"), 0)
)

# Validate data
df = df.filter(col("amount") >= 0)

# Cache if reusing
df.cache()

# Operations
result = df.groupBy("status").count()
result.show()

# Save with error handling
try:
    result.write \
        .mode("overwrite") \
        .csv("output/result")
    print("Saved successfully!")
except Exception as e:
    print(f"Save error: {e}")

spark.stop()

except Exception as e:
    print(f"Pipeline error: {e}")

```

└─ Run: python3 optimized_pipeline.py
└─ Result: Error handling working 

Task 3: Add Logging (30 mins)

└─ Create: logged_pipeline.py

└─ Code:

```

import logging
from pyspark.sql import SparkSession

```

```

# Setup logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler('pipeline.log'),
        logging.StreamHandler()
    ]
)

logger = logging.getLogger(__name__)

try:
    logger.info("Starting pipeline...")

    spark = SparkSession \
        .builder \
        .appName("LoggedPipeline") \
        .master("local[*]") \
        .getOrCreate()

    logger.info("Reading data...")
    df = spark.read \
        .option("header", "true") \
        .option("inferSchema", "true") \
        .csv("sample_data.csv")

    logger.info(f"Records read: {df.count()}")

    logger.info("Processing data...")
    result = df.groupBy("status").count()

    logger.info("Saving results...")
    result.write \
        .mode("overwrite") \
        .csv("output/result")

    logger.info("Pipeline completed successfully!")

except Exception as e:
    logger.error(f"Pipeline failed: {e}")
finally:
    spark.stop()
    logger.info("Spark session closed")

```

└─ Run: python3 logged_pipeline.py

└─ Result: Logs created in pipeline.log 


```
Task 4: Performance Monitoring (20 mins)
├─ Learn: Spark UI monitoring
├─ Run: python3 logged_pipeline.py
├─ While running:
│   └─ Access: http://localhost:4040
│   └─ View: Job progress, stage details, executor information
└─ Understand: Performance metrics
```

```
Task 5: Commit & Document (5 mins)
└─ Commit: "Day 13: Error handling & logging added"
```

LEARNING OBJECTIVES:

-  Error handling with try-except
-  Logging setup
-  Null value handling
-  Performance monitoring
-  Production-ready code

DELIVERABLE: Robust pipeline with logging

DAY 14 (June 14) - WEEK 2 REVIEW & FINALIZE

Duration: 120 mins

MORNING TASK:

```
Task 1: Test & Verify Pipeline (30 mins)
├─ Test: With large dataset
├─ Verify: All outputs created
├─ Check: Data quality
├─ Measure: Execution time
└─ Document: Results
```

```
Task 2: Code Review & Cleanup (30 mins)
├─ Review: All Python files
├─ Add: Comments to code
├─ Organize: Project structure
├─ Create: README for pipeline
└─ Format: Professional code
```

```
Task 3: Create Documentation (35 mins)
├─ Create: README.md
├─ Content:
│   └─ # Sales Pipeline
```

```
## Overview
Process sales data with Spark
```

Requirements

- PySpark 3.x
- Python 3.7+

Usage

```
python3 sales_pipeline.py
```

Output Files

- output/sales_by_status/
- output/sales_by_product/

Data Schema

```
[List all columns and types]
```

- └─ Create: SETUP.md
- └─ Content: Installation steps
- └─ Create: ARCHITECTURE.md

Task 4: Final Testing (20 mins)

- └─ Run: Complete pipeline
- └─ Verify: All outputs correct
- └─ Test: Error handling
- └─ Time: Execution time
- └─ Save: Results to S3

Task 5: Commit & Archive (5 mins)

- └─ Create: WEEK2_SUMMARY.md
- └─ Recap: What learned
- └─ List: Files created
- └─ Record: Status
- └─ Commit: "Day 14: Week 2 complete! Spark pipeline working perfectly!"

WEEK 2 CHECKPOINT:

- ✅ Spark DataFrames created
- ✅ Transformations working
- ✅ Aggregations implemented
- ✅ Error handling added
- ✅ Logging setup
- ✅ Documentation complete
- ✅ 30 commits total (Days 1-14)

Status: 🟢 Ready for Docker!

LEARNING OBJECTIVES:

-  Consolidated Week 2 learning
-  Professional code standards
-  Complete documentation

DELIVERABLE: Production-ready Spark pipeline

WEEK 3 (June 15-21): DOCKER & CONTAINERIZATION

DAY 15 (June 15) - DOCKER BASICS

Duration: 120 mins

MORNING TASK:

Task 1: Understand Docker (25 mins)

- |— Concept: Containerization
- |— Benefit: Portability, consistency
- |— Components: Images, Containers, Registry
- |— Learn: Docker basics

Task 2: Install Docker (25 mins)

- |— SSH into EC2
- |— Command: `sudo apt-get install docker.io -y`
- |— Command: `sudo usermod -aG docker ubuntu`
- |— Command: `docker --version`
- |— Verify: Docker installed 

Task 3: Learn Docker Commands (30 mins)

- |— `docker images` (list images)
- |— `docker run` (run container)
- |— `docker ps` (list running containers)
- |— `docker stop` (stop container)
- |— `docker logs` (view logs)
- |— `docker rm` (remove container)
- |— Practice: Each command multiple times

Task 4: Create Dockerfile (35 mins)

- |— Create: Dockerfile (no extension)
- |— Content:

```
# Use Python base image
FROM python:3.9-slim

# Set working directory
WORKDIR /app
```

```
# Copy requirements
COPY requirements.txt .

# Install dependencies
RUN pip install -r requirements.txt

# Copy application code
COPY . .

# Set environment variables
ENV PYTHONUNBUFFERED=1

# Run pipeline
CMD ["python3", "sales_pipeline.py"]
```

└─ Create: requirements.txt

└─ Content:

pyspark==3.5.0

boto3==1.26.0

pandas==1.5.0

numpy==1.24.0

└─ Save: Both files

Task 5: Test Dockerfile (5 mins)

└─ Verify: Files in correct location

└─ Ready: For build tomorrow

└─ Commit: "Day 15: Dockerfile created"

LEARNING OBJECTIVES:

-  Docker concepts understood
-  Docker installed
-  Docker commands practiced
-  Dockerfile created
-  requirements.txt ready

DELIVERABLE: Dockerfile + requirements.txt

DAY 16 (June 16) - BUILD DOCKER IMAGE

Duration: 120 mins

MORNING TASK:

Task 1: Build Docker Image (40 mins)

Task 1: Build Docker Image (40 mins)

- SSH into EC2
- Navigate: `cd ~/data-engineering-project`
- Command: `docker build -t sales-pipeline:v1 .`
- Wait: Build completes (5-10 mins)
- Verify: "Successfully tagged sales-pipeline:v1"
- Result: Image created 

Task 2: Verify Image (20 mins)

- Command: `docker images`
- Check: sales-pipeline:v1 appears in list
- Command: `docker image inspect sales-pipeline:v1`
- View: Image details (size, layers, etc.)
- Result: Image verified 

Task 3: Test Image Locally (40 mins)

- Command: `docker run -it sales-pipeline:v1 /bin/bash`
- Inside container:
 - Command: `pwd` (verify /app)
 - Command: `ls -la` (see files)
 - Command: `python3 --version`
 - Command: `python3 -c "import pyspark; print(pyspark.__version__)"`
 - Command: `exit` (exit container)
- Result: Container working 

Task 4: Run Pipeline in Container (15 mins)

- Command: `docker run -v $(pwd)/output:/app/output sales-pipeline:v1`
- Wait: Pipeline runs
- Check: Output files created
- Result: Container execution successful 

Task 5: Commit & Document (5 mins)

- Update: `day_16_notes.md`
- Record: Image size, layers
- Commit: "Day 16: Docker image built and tested"

LEARNING OBJECTIVES:

-  Docker image built successfully
-  Image tested locally
-  Pipeline runs in container
-  Volume mounting working
-  Container execution verified

DELIVERABLE: Working Docker image

DAY 17 (June 17) - DOCKER HUB & REGISTRY

Duration: 120 mins

MORNING TASK:

Task 1: Create Docker Hub Account (15 mins)

- └─ Go to: `hub.docker.com`
- └─ Sign up: Create free account
- └─ Verify: Email verified
- └─ Login: Docker Hub ready
- └─ Result: Account created 

Task 2: Push Image to Docker Hub (45 mins)

- └─ SSH into EC2
- └─ Command: `docker login`
- └─ Enter: Docker Hub username
- └─ Enter: Docker Hub password/token
- └─ Verify: "Login Succeeded"
- └─ Command: `docker tag sales-pipeline:v1 YOUR_USERNAME/sales-pipeline:v1`
- └─ Command: `docker tag sales-pipeline:v1 YOUR_USERNAME/sales-pipeline:latest`
- └─ Command: `docker push YOUR_USERNAME/sales-pipeline:v1`
- └─ Wait: Upload completes
- └─ Command: `docker push YOUR_USERNAME/sales-pipeline:latest`
- └─ Wait: Upload completes
- └─ Result: Images on Docker Hub 

Task 3: Verify on Docker Hub (20 mins)

- └─ Go to: `hub.docker.com`
- └─ Find: Your repository
- └─ Check: Tags (v1 and latest)
- └─ Copy: Pull command
- └─ Record: Registry URL

Task 4: Pull and Test (35 mins)

- └─ Command: `docker rmi YOUR_USERNAME/sales-pipeline:v1` (remove local)
- └─ Command: `docker pull YOUR_USERNAME/sales-pipeline:v1` (pull from hub)
- └─ Wait: Download completes
- └─ Command: `docker run YOUR_USERNAME/sales-pipeline:v1`
- └─ Verify: Pipeline runs from registry
- └─ Result: Pull working 

Task 5: Document & Commit (5 mins)

- └─ Update: `day_17_notes.md`
- └─ Record: Docker Hub URL

```
└─ Commit: "Day 17: Image on Docker Hub! Pushed successfully"
```

LEARNING OBJECTIVES:

-  Docker Hub account created
-  Image tagged properly
-  Image pushed to Docker Hub
-  Image pulled from registry
-  Registry workflow understood

DELIVERABLE: Docker image on Docker Hub

DAY 18 (June 18) - AWS EC2 DEPLOYMENT PREP

Duration: 120 mins

MORNING TASK:

Task 1: Create New EC2 for Production (30 mins)

```
└─ AWS Console → EC2 → Instances
└─ Launch Instance:
    └─ Image: Ubuntu 20.04 LTS
    └─ Type: t2.micro (free tier)
    └─ Name: data-eng-production
    └─ Key Pair: Use existing or create new
    └─ Security: Allow SSH (22) + Custom TCP (8080 for logs)
    └─ Storage: 50GB
└─ Launch instance
└─ Wait: Status "running"
└─ Note: Public IP address
```

Task 2: Setup Production Instance (40 mins)

```
└─ SSH: ssh -i keypair.pem ubuntu@PUBLIC_IP
└─ Update system:
    └─ sudo apt-get update
    └─ sudo apt-get upgrade -y
    └─ sudo apt-get install docker.io -y
    └─ sudo usermod -aG docker ubuntu
└─ Verify: docker --version
└─ Result: Docker ready on production 
```

Task 3: Create Output Directory (20 mins)

```
└─ Command: mkdir -p ~/pipeline/output
└─ Command: mkdir -p ~/pipeline/logs
└─ Command: mkdir -p ~/pipeline/data
```

└─ Command: mkdir -p ~/pipeline/data

└─ Structure: Ready for outputs

└─ Result: Directories created 

Task 4: Test Docker on Production (25 mins)

└─ Command: docker pull YOUR_USERNAME/sales-pipeline:latest

└─ Wait: Image downloads

└─ Command: docker run -v ~/pipeline/output:/app/output YOUR_USERNAME/sales-pipeline:latest

└─ Check: ~/pipeline/output/ for results

└─ Result: Pipeline works on production 

Task 5: Commit & Document (5 mins)

└─ Commit: "Day 18: Production EC2 ready, Docker tested"

LEARNING OBJECTIVES:

-  Production EC2 instance created
-  Docker installed on production
-  Container runs on production
-  Output storage ready
-  Deployment workflow tested

DELIVERABLE: Production-ready EC2 instance

DAY 19 (June 19) - AUTOMATE & SCHEDULE

Duration: 120 mins

MORNING TASK:

Task 1: Create Cron Job (35 mins)

└─ SSH into production instance

└─ Command: crontab -e

└─ Add cron job for daily execution:

0 2 * * * docker run -v ~/pipeline/output:/app/output YOUR_USERNAME/sales-pipeline:latest >> ~/pipeline/logs/cron.log 2>&1

|

| Cron format: minute hour day month weekday command

| 0 2 * * * = Every day at 2 AM

|

└─ Save cron job

└─ Verify: crontab -l

└─ Result: Automated execution 

Task 2: Monitor Pipeline Execution (30 mins)

Task 2: Monitor Pipeline Execution (30 mins)

- Command: `docker run -d YOUR_USERNAME/sales-pipeline:latest`
(Run in background)
- Command: `docker ps` (see running containers)
- Command: `docker logs CONTAINER_ID` (view logs)
- Monitor: Execution progress
- Check: Output files created

Task 3: Create Health Check (35 mins)

- Create: `health_check.sh`
- Content:

```
#!/bin/bash

# Check if pipeline ran today
if [ -f ~/pipeline/output/sales_by_status/part-*.csv ]; then
    echo "Pipeline ran successfully today"
    exit 0
else
    echo "Pipeline failed or not run yet"
    exit 1
fi
```

- Make executable: `chmod +x health_check.sh`
- Run: `./health_check.sh`
- Result: Health check working 

Task 4: Setup CloudWatch Logs (15 mins)

- SSH to production
- Create log file: `~/pipeline/logs/pipeline.log`
- Run pipeline with logging:
`docker run -v ~/pipeline/output:/app/output -v ~/pipeline/logs:/app/logs`
`YOUR_USERNAME/sales-pipeline:latest`
- Verify: Logs created 

Task 5: Commit & Document (5 mins)

- Commit: "Day 19: Automated scheduling with cron"

LEARNING OBJECTIVES:

-  Cron job created for automation
-  Health check script working
-  Logs monitored
-  Automated daily execution
-  Production ready

DELIVERABLE: Automated pipeline execution

DAY 20 (June 20) - MONITORING & LOGGING

Duration: 120 mins

MORNING TASK:

Task 1: Enhance Logging (30 mins)

└─ Update: sales_pipeline.py

└─ Add structured logging:

```
import json
import logging
from datetime import datetime

# Setup structured logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
)
logger = logging.getLogger('SalesPipeline')

def log_event(event_type, details):
    log_entry = {
        'timestamp': datetime.now().isoformat(),
        'event': event_type,
        'details': details
    }
    logger.info(json.dumps(log_entry))
```

└─ Result: Structured logging 

Task 2: Create Monitoring Dashboard (35 mins)

└─ Create: monitor.py

└─ Code:

```
import os
import json
from pathlib import Path

def get_pipeline_stats():
    output_dir = Path('output')

    stats = {
        'last_run': None,
        'total_records': 0,
        'total_sales': 0,
        'status_breakdown': {}
    }
```

```
# Check output files
for csv_file in output_dir.rglob('*.csv'):
    stats['last_run'] = os.path.getmtime(csv_file)

return stats
```

```
# Run monitoring
stats = get_pipeline_stats()
print(json.dumps(stats, indent=2, default=str))
```

- └─ Create: metrics.json
- └─ Update: After each run

Task 3: Create Alert System (30 mins)

- └─ Create: alerts.py
- └─ Code:

```
import json
from datetime import datetime, timedelta

def check_pipeline_health():
    # Check if pipeline ran in last 24 hours
    # Check if output files exist
    # Check file sizes

    alerts = []

    # Example: Check file size
    output_file = 'output/sales_by_status/part-00000.csv'
    if os.path.getsize(output_file) == 0:
        alerts.append({
            'level': 'ERROR',
            'message': 'Output file is empty',
            'timestamp': datetime.now().isoformat()
        })

    return alerts
```

- └─ Run: python3 alerts.py
- └─ Result: Alert system ready 

Task 4: Email Notifications (20 mins)

- └─ Create: notify.py
- └─ Code (placeholder):

```
import smtplib
from email.mime.text import MIMEText
```

```
def send_alert(subject, message):  
    # Setup email configuration  
    # Send notification on pipeline failure  
    pass
```

└─ Note: Implement with your email credentials

Task 5: Commit & Documentation (5 mins)

└─ Commit: "Day 20: Monitoring, logging, and alerts setup"

LEARNING OBJECTIVES:

-  Structured logging implemented
-  Monitoring dashboard created
-  Alert system setup
-  Health checks automated
-  Production monitoring ready

DELIVERABLE: Complete monitoring system

DAY 21 (June 21) - WEEK 3 REVIEW & POLISH

Duration: 120 mins

MORNING TASK:

Task 1: Test Complete System (40 mins)

└─ Test: Full pipeline end-to-end
└─ Verify: Outputs created
└─ Check: Logs generated
└─ Monitor: Health status
└─ Time: Execution time
└─ Document: Results

Task 2: Code Cleanup & Documentation (40 mins)

└─ Review: All Python files
└─ Add: Comprehensive comments
└─ Create: API documentation
└─ Document: Configuration options
└─ Create: Troubleshooting guide
└─ Professional: Code standards

Task 3: Security Review (20 mins)

└─ Check: No hardcoded credentials
└─ Verify: IAM role proper
└─ Secure: Environment variables used

```
| Secure: Environment variables |  
| Review: Access controls |  
| Secure: S3 bucket permissions |
```

Task 4: Create Deployment Guide (15 mins)

```
| Create: DEPLOYMENT.md |  
| Include:
```

- Prerequisites
- Step-by-step setup
- Configuration
- Running pipeline
- Troubleshooting

```
| Professional: Complete documentation |
```

Task 5: Final Commit & Summary (5 mins)

```
| Create: WEEK3_SUMMARY.md |  
| Record: Everything completed |  
| Commit: "Day 21: Week 3 complete! Docker & Deployment ready!" |
```

WEEK 3 CHECKPOINT:

- ✓ Docker containerized
- ✓ Pushed to Docker Hub
- ✓ Running on production EC2
- ✓ Automated with cron
- ✓ Logging & monitoring
- ✓ Alert system ready
- ✓ Documentation complete

Status:  Production ready!

LEARNING OBJECTIVES:

- ✓ Consolidated Week 3 learning
- ✓ Professional documentation
- ✓ Production deployment
- ✓ Automation setup

DELIVERABLE: Production-ready containerized pipeline

WEEK 4 (June 22-30): POLISH & DOCUMENTATION

DAY 22-25 (June 22-25) - CODE POLISH & DOCUMENTATION

Duration: 120 mins × 4 days

Daily Tasks:

Day 22: Code refactoring

- Optimize Spark queries
- Improve error handling
- Add type hints
- Professional formatting

Day 23: Create comprehensive docs

- README.md
- SETUP.md
- API documentation
- Configuration guide

Day 24: Add advanced features

- Data validation
- Logging to CloudWatch
- Metrics collection
- Performance tuning

Day 25: Final code review

- Security audit
 - Performance check
 - Test coverage
 - Production readiness
-

DAY 26-28 (June 26-28) - BLOG & MEDIUM POST

Duration: 120 mins × 3 days

Day 26: Write blog outline

- Title: "Building a Production Data Pipeline with Spark and AWS"
- Sections: Architecture, Implementation, Deployment
- Code examples ready

Day 27: Write detailed blog

- Introduction & motivation
- Architecture diagram description
- Code walkthroughs

- Challenges & solutions
- Performance metrics

Day 28: Publish on Medium

- Polish writing
 - Add code blocks
 - Format properly
 - Publish
 - Share on social media
-

DAY 29-30 (June 29-30) - FINAL POLISH

Duration: 120 mins × 2 days

Day 29: Final testing & optimization

- Run complete pipeline
- Verify all outputs
- Measure performance
- Document results

Day 30: Archive & summary

- Create comprehensive README
- Document all learnings
- Create final summary
- Prepare for July phase

JUNE 30 FINAL CHECKPOINT:

✅ PHASE 1 COMPLETE!

DATA ENGINEERING ACHIEVEMENTS:

- ✅ Spark pipeline built & tested
- ✅ Docker image created & pushed
- ✅ EC2 deployment working
- ✅ Automated scheduling setup
- ✅ Monitoring & alerts ready
- ✅ Blog post published
- ✅ Complete documentation

STATISTICS:

- Days completed: 30

- Lines of code: 1000+
- Technologies: Spark, Docker, AWS
- Output files: 3+ CSV reports
- GitHub commits: 30
- Blog views: TBD

STATUS: 🏆 Ready for Phase 2!

⚡ PHASE 2: JULY (Days 31-61) - OPTIMIZATION & MASTERY

(Similar detailed breakdown with focus on:

- Performance optimization
 - Advanced Spark features
 - AWS cost optimization
 - System scaling
 - Production hardening
 - Team documentation)
-

🇺🇸 COMPLETE DATA ENGINEERING CHECKLIST

By August 1:



CORE SKILLS

- ☐ Spark fundamentals mastered
- ☐ AWS EC2, S3, CloudWatch expert
- ☐ Docker containerization expert
- ☐ Data pipeline architecture understanding
- ☐ Python for data engineering fluent



PROJECTS

- ☐ Sales pipeline deployed to production
- ☐ Docker image on Docker Hub
- ☐ Monitoring system setup
- ☐ Automated daily execution
- ☐ Blog post published



PORTFOLIO

- ☐ GitHub repository professional
- ☐ 60+ commits over 61 days
- ☐ Complete documentation

- ☐ Real-world project showcase
- ☐ Medium article with 1000+ views

✅ PRODUCTION READY

- ☐ Code optimized
- ☐ Error handling robust
- ☐ Logging comprehensive
- ☐ Monitoring active
- ☐ Security hardened

FINAL STATUS: 🏆 DATA ENGINEER READY!

This is your detailed day-wise data engineering roadmap for 61 days! Follow it day by day, and you'll be a cloud data engineer by August 1! 🚀